

Python 01: working with text

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM11.html

Quick links:
Click on any  to get back here!

- It's an interpreter
 - Starting Python on the Bioinformatics servers
 - creating an alias (optional!)
 - Statements, functions, arguments: parts of code
 - Using quotes
 - Using comments
 - Special characters: escaping
 - Variables in Python
 - Multiple assignments
 - String concatenation
 - String length `len()`
 - Numbers aren't strings
 - Converting between integers and strings : `str()`
 - Converting between integers and strings : `int()`
 - Changing case: `upper()` , `lower()`
 - Find and replace in strings: `replace()`
 - Extracting part of a string: `[0:4]`
 - Counting substrings: `count()`
 - Finding positions of substrings: `find()`
 - Summary of things!
-
- ### Exercises
1. Calculating A+T content
 2. Complementing DNA
 3. Restriction fragment lengths
 4. Splicing out introns
-
- important git/GitHub stuff at the end

Goals for this bit of the course

1. introduce many core Python features
2. as much hands-on time as possible
3. solving biological problems
4. useful things first: if it isn't useful to you, you won't use it, and then you'll forget it...

How to get the most out of the exercises

- listen then do!
- exercise descriptions are often (deliberately) vague!
- break exercises down into small chunks and test
- substitute with problems/data from your own research interests if you want
- these sessions are based on these books...
 - [Python for Biologists by Martin Jones](#)
 - [Advanced Python for Biologists by Martin Jones](#)

What I don't intend to cover

- installing Python on your favourite operating system (too many variables, and Google **IS** your friend!): go to <https://www.python.org/>
- using a GUI (yes, I'm old-fashioned, I know, sorry no [PyCharm](#) or [IDLE](#) or [Jupyter Notebook](#) or similar!)
- laptop troubleshooting
- exhaustive lists of features
- different versions/implementations
- optimizing for speed/memory

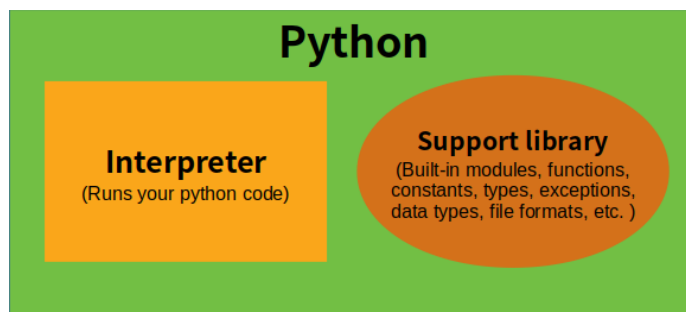
Why Python?

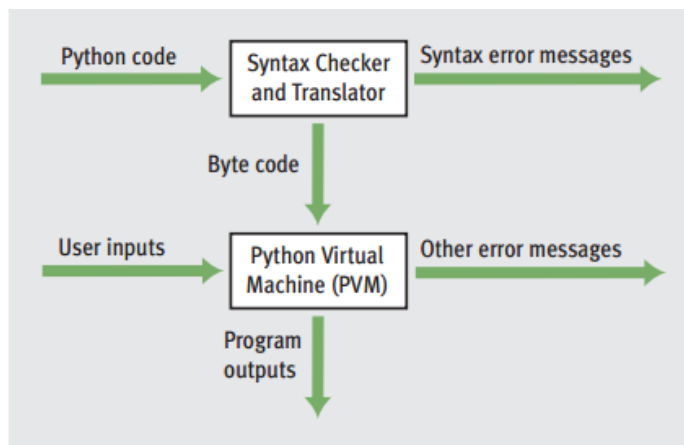
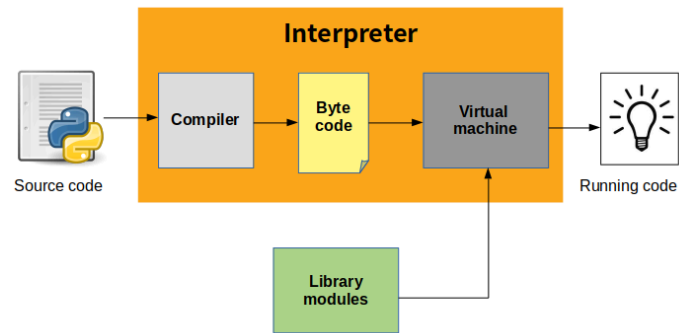
- consistent syntax
- built-in libraries
- one obvious way of doing things (usually!)
- widely used
- shallow learning curve
- very readable
- used in science a lot
- data libraries
- good with text (DNA/protein sequences etc)

🏠 Remember this from our very first lecture? [01 What is a computer](#)

The Python high-level language uses an **interpreter**.

An interpreter translates and immediately executes the high-level language instructions we write (see [more here](#))





Assembler: programme which translates assembly language to machine language format (object file) for execution

Compiler: translates a high-level language programme into a saved machine language (object) programme, which can then be executed at any time in the future: compile once, run often!

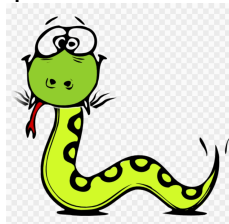
Interpreter: programme that directly executes instructions written in a programming or scripting language, without previously compiling them into a machine language programme

The Python interpreter can be used in two ways:

- interactive mode: we type Python statements on the keyboard
- script mode: for example, we have a `myscript.py` file that contains Python statements that we tell the interpreter to process

... this is all part of the fun that awaits us when we start on Python!

Well, now we have got to that point....!



📌 Starting Python on the Bioinformatics servers

Where is our python software on [msc8](#)?

which python

/usr/bin/python

python --version

Python 3.12.3

which python3

/usr/local/bin/python3

python3 --version

Python 3.12.3

Where is our python software on [msc7](#)?

which python

/usr/bin/python

python --version

Python 3.12.3

which python3

/usr/local/bin/python3

python3 --version

Python 3.12.3

Some servers might have different versions, possibly including python 2.7.xx
... we definitely want to use Python 3

Make sure you are using Python 3!

```
# We could make ourselves a symbolic link to make sure  
# it's always python3 we get
```

```
cd
```

```
ln -s /usr/bin/python3 p3
```

```
./p3
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
...
```



```
# We could make an alias for it when we log in, and  
# it will work on all our servers
```

```
# First, remove the symbolic link we just made
```

```
rm p3
```

```
# Add a line to our .bash_profile file
```

```
echo "alias p3= \$(which python3)" >> ${HOME}/.bash_profile
```

```
# Simulate logging in
```

```
source ${HOME}/.bash_profile
```

```
# Check it as an alias
```

```
alias p3
```

```
alias p3='/usr/bin/python3'
```

```
# Check it as a variable
```

```
echo ${p3}
```

```
# See if it works
```

```
p3
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>>  
CTRL-D to exit
```

Now, we can (if we want to, entirely up to you...!) use the alias command `p3` to

ALWAYS start Python3 irrespective of which server we have logged on to...!

p3

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

help

```
Type help() for interactive help, or help(object) for help about object.  
>>>
```

From now on, I am not going to show the `>>>` python prompt...

help()

Welcome to Python 3.12's help utility! If this is your first time using Python, you should definitely check out the tutorial at <https://docs.python.org/3.12/tutorial/>.

Enter the name of any module, keyword, or topic to get help on writing Python programs and using Python modules. To get a list of available modules, keywords, symbols, or topics, enter "modules", "keywords", "symbols", or "topics".

Each module also comes with a one-line summary of what it does; to list the modules whose name or summary contain a given string such as "spam", enter "modules spam".

To quit this help utility and return to the interpreter, enter "q" or "quit".

help> q

You are now leaving help and returning to the Python interpreter. If you want to ask for help on a particular object directly from the interpreter, you can type "help(object)". Executing "help('string')" has the same effect as typing a particular string at the help> prompt.

📌 Statements, functions, arguments: parts of code

Start with a very short Python program! First, a few words about comments put in Python code. There are several alternatives, which include the following:

This is a comment, one hash sign per line

''' This triple quote version (single or doubles, but NOT a mixture!) can also be used over several lines, as long as it is closed by the same thing ''' as well (i.e. three singles or three doubles).

These can also have a different meaning (which we won't go into here!) to Python, so if you want to know more, [read this!](#)

Does this work like it would in awk?

print "Hello World"

```
File "<stdin>", line 1
  print "Hello World"
    ^
```

SyntaxError: Missing parentheses in call to 'print'. Did you mean print("Hello World")?

Does this work like it would in bash?

echo "Hello World"

```
File "<stdin>", line 1
  echo "Hello World"
    ^
```

SyntaxError: invalid syntax

How do we do it in Python?

print("Hello World")

Hello World

print("Hello World") ;

Hello World

print is the name of a function.

"Hello World" is the function arguments.

print("Hello World") is a statement (one instruction).

🏠 'To quote or not to quote' (or "two quote or not two quote")

(with apologies to [Shakespeare](#) for plagiarism)

Things in quotes are strings (of characters) - we can use either a single or a double quote:

Single quote

```
print('Hello world')
```

```
Hello world
```

Double quotes

```
print("Hello world")
```

```
Hello world
```

Both together

```
print("Hamish said, 'Hello world'")
```

```
Hamish said, 'Hello world'
```

But you can't combine the two in an unbalanced way

```
print("Hello world')
```

```
File "<stdin>", line 1
  print("Hello world')
                        ^
SyntaxError: EOL while scanning string literal
```

📌 Using comments: just as important for Python scripts!

Lines of text meant for humans, not computers, to read, start with a `#`, as we saw previously.

A **comment** is where I generally put information about what the code is designed to do, and are usually at the beginnings of lines.

Comments are also a good place to put "notes to self".

If you have these in a defined format, then you can `grep` them out later....

```
# print a friendly greeting
```

```
# print("Hello world")
```

```
# NOTE_01_DETAILS: "Hello world" is getting pretty tedious, maybe  
implement something else???
```

```
# NOTE_01_STATUS: COMPLETED 04 Oct 2025
```

```
# NOTE_02_DETAILS: Look into making the greeting user-specific, maybe get  
# user name from the login with "echo $USER"?
```

```
# NOTE_02_STATUS: OPEN 21 Oct 2025
```

```
print("Hello class") # I changed this
```

```
Hello class
```

If we have written 5,000 lines of code for a particular project, all in a file called `myscript.py`, how could we check that all "notes" had been dealt with?

```
# Run in a Linux terminal: case insensitive...
```

```
grep -ic "status: open" myscript.py
```

```
some number
```

📌Special characters

Some characters in Python have a special meaning.

"String literals" can be enclosed in matching single quotes `'` or double quotes `"`.

They can also be enclosed in matching groups of three single or double quotes (these are generally referred to as **triple-quoted strings**).

The backslash `\` character is used to **"escape"** characters that otherwise have a special meaning, such as newline, backslash itself, or the quote character.

Coded	Meaning
<code>\\</code>	Backslash <code>\</code>
<code>\'</code>	Single quote <code>'</code>
<code>\"</code>	Double quote <code>"</code>
<code>\a</code>	ASCII Bell (BEL)
<code>\b</code>	ASCII Backspace (BS)
<code>\r</code>	ASCII Carriage Return (CR)
<code>\n</code>	ASCII Linefeed (LF)
<code>\f</code>	ASCII Formfeed (FF)
<code>\t</code>	ASCII Horizontal Tab (TAB)
<code>\v</code>	ASCII Vertical Tab (VT)

Err... carriage return, line feed, etc., aren't they all the same??

EOL (end of line), CR and LF are ASCII representations for the end of the line, but they are all interpreted in different ways by different operating systems. [\[History of all this, if you are interested!\]](#)

- The Carriage Return (CR) character (0x0D, `\r`) moves the cursor to the beginning of the line without advancing to the next line. This character is used as a new line character in Commodore and early Macintosh operating systems (OS-9 and earlier).
- The Line Feed (LF) character (0x0A, `\n`) moves the cursor down to the next line without returning to the beginning of the line first. This character is used as a new line character in Linux based systems (Linux, Mac OSX, etc)
- The End of Line (EOL) character (0x0D0A, `\r\n`) is actually two ASCII characters and is a combination of the CR and LF characters. It moves the cursor both down to the next line and to the beginning of that line. This character is used as a new line character in most other non-Linux operating systems including Microsoft Windows, Symbian OS and others.
- The Form Feed (FF) character (0x0C, `\f`) means advance downward to the next "page". It was commonly used as page separators, but now can also be used as section separators. (It's uncommonly used in source code to divide logically independent functions or groups of functions.) Text editors can use this character when you "insert a page break".

```
# Print the greeting over two lines
```

```
print("Hello\nclass")
```

```
Hello  
class
```

**# Print the greeting over two lines, with a
tab indent on the second line**

```
print("Hello\n\tclass")
```

```
Hello
      class
```

Print the greeting with a vertical tab

```
print("Hello\vclass")
```

```
Hello
      class
```

**# Print the greeting over two lines, with an
incorrectly formatted tab indent on the second line**

```
print("Hello\ntclass")
```

```
Hello
tclass
```

**# Print the greeting over two lines, using
some other special characters**

```
print("Hello | class \n\t>$ how are you doing \\\ \"")
```

```
Hello | class
      >$ how are you doing \\ ?
```

**# Print the greeting over two lines,
using the carriage return and a tab**

```
print("Hello | \rclass \t>$ how are you doing \\\ \"")
```

```
class | >$ how are you doing \\ ?
```

**# Print the greeting over two lines,
using the carriage return but NO tab**

```
print("Hello | \rclass >$ how are you doing \\\ \"")
```

```
class >$ how are you doing \\ ?
```

**# Print the greeting over two lines,
using the form feed to indent**

```
print("Hello | \fclass \f>$ how are you \fdoing \\\ ?")
```

```
Hello |  
    [new page]  
    class  
    [new page]  
    >$ how are you  
    [new page]  
    doing \ \ ?
```

In the above examples, what we used as pipe commands earlier in the course are not read as pipe commands here: why might that be?

Did you notice how we **escaped** the backslashes?

🏠 Variables

- To store a bit of information to use later, we use a **variable**, similar to what we have seen before in **bash** or **awk**.
- Variables are reserved memory locations to store values. When you create a variable you reserve some space in memory; turn the computer off and they are gone.
- Based on the **data type** of a variable, the command line interpreter allocates memory and decides what can be stored in the reserved memory.
- By assigning different data types to variables, you can store integers, decimals or characters in these variables.

Variables: with examples

- Python variables do not need explicit declaration to reserve memory space: declaration happens automatically when you assign a value to a variable.
- The equals sign `=` is used to assign values to variables.
- The **operand** to the left of the `=` operator is the **name** of the variable and the **operand** to the right of the `=` operator is the **value** stored in the variable.

In Python, you **can** have a space next to the equals sign, yaay!!

```
counter = 100 # An integer assignment
```

```
miles = 1000.0 # A floating point
```

```
girlsname = "Stella" # A text string
```

Here, 100, 1000.0 and "Stella" are the values assigned to `counter`, `miles`, and `girlsname` variables, respectively.

```
print("counter")
```

```
counter
```

```
print(counter)
```

```
100
```

```
# Add a space to the command, it's OK
```

```
print (counter)
```

```
100
```

Add several spaces, still OK

```
print ( counter )
```

```
100
```

```
print(miles)
```

```
1000.0
```

```
print(girlsname)
```

```
Stella
```

```
print(counter, miles, girlsname)
```

```
100 1000.0 Stella
```

Notice the spaces in the output!

```
print(counter, "\n", miles, "\n", girlsname)
```

```
100
```

```
1000.0
```

```
Stella
```

📌 Multiple Assignments 1

Python allows you to assign a single value to several variables simultaneously.

```
a = b = c = 1
```

```
print a
```

```
File "<stdin>", line 1
  print a
    ^
SyntaxError: Missing parentheses in call to 'print'
```

```
print(a)
```

```
1
```

```
print(b)
```

```
1
```

```
print(c)
```

```
1
```

Here, an **integer** object is created with the value 1, and all three variables are assigned to the same memory location.

Multiple Assignments 2

Python also allows you to assign multiple objects to multiple variables.

```
a, b, c = 1, 2.805, "snake"
```

Here,

- an **integer** with value 1 is assigned to variable `a`
- a **floating point** object with value 2.805 is assigned to variable `b`
- one **string** object with the value "snake" is assigned to the variable `c`

```
print(a)
```

```
1
```

```
print(b)
```

```
2.805
```

```
print(c)
```

```
snake
```

Let's make this more relevant!

```
# store a DNA sequence in the variable my_dna
```

```
my_dna = "ATGCGTA"
```

```
# now print the DNA sequence
```

```
print(my_dna)
```

```
ATGCGTA
```

```
# change the value of my_dna
```

```
my_dna = "TGGTCCA"
```

```
# print it again
```

```
print(my_dna)
```

```
TGGTCCA
```



Variable names can be whatever we want - this code is exactly the same as the previous one:

```
# store a DNA sequence in a variable called hedgehog
```

```
hedgehog = "ATGCGTA"
```

```
# now print the DNA sequence
```

```
print(hedgehog)
```

```
ATGCGTA
```

```
# change the value of hedgehog
```

```
hedgehog = "TGGTCCA"
```

```
# print it again
```

```
print(hedgehog)
```

```
TGGTCCA
```

```
# print both variables at the same time with some text in between
```

```
print( hedgehog , "is the same as" , my_dna )
```

```
TGGTCCA is the same as TGGTCCA
```

📌 Manipulating text strings: concatenation

```
# We can join two strings together with +
```

```
"AANNNTT" + "GGCC"
```

```
AANNNTTGGCC
```

```
# We can do sums with numbers
```

```
4 * 6
```

```
24
```

```
4 * 3.9
```

```
15.6
```

```
# We can even multiply strings...
```

```
"ACTG" * 3
```

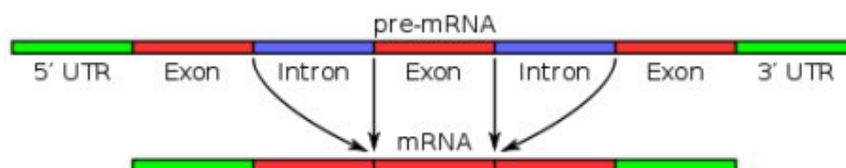
```
'ACTGACTGACTG'
```

```
# ...but in sensible ways only!
```

```
"ACTG" * 3.25
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: can't multiply sequence by non-int of type 'float'
```

Anywhere we can use a string written with quotes, we can also use a variable that stores a string



[link](#)

```
upstream = "AAA"
```

```
upstream
```

```
'AAA'
```

```
my_dna = upstream + "ATGC"
```

```
print(my_dna)
```

```
AAAATGC
```

```
my_dna
```

```
'AAAATGC'
```

```
upstream = "AAA"
```

```
downstream = "ggg"
```

```
my_dna = upstream + "ATGC" + downstream
```

```
my_dna
```

```
'AAAATGCggg'
```

```
# The result of concatenating two (or more) strings is also a string:
```

```
print("Hello" + " " + "class")
```

```
Hello class
```

📌 Manipulating text strings: lengths

To find the length of a string, use the `len()` function:

```
len("AGTC")
```

```
4
```

```
len("123")
```

```
3
```

```
len(123)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: object of type 'int' has no len()
```

What we need to know about `len()`:

1. `len()` is a function
2. it takes one argument
3. the argument has to be a string
4. it returns the length of the string, "number of items of a sequence or collection"
5. we have to do something with the returned value
6. the length is a number, not a string!

📌 Numbers aren't strings

Numbers and strings are different:

```
2 + 2
```

```
4
```

```
"abc" + "def"
```

```
'abcdef'
```

We can't add numbers and strings, even if we can multiply them!

```
2 + "abc"
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

We can't subtract strings either

```
"hello" - "o"
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

We already know numbers don't have lengths

```
len(123)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: object of type 'int' has no len()
```

📌 Converting between integers and strings : str()

```
4 + "abc"
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

`str()` will take an integer (or float) and turn it into a string

```
str(4) + "abc"
```

```
'4abc'
```

```
str(4.99) + "abc"
```

```
'4.99abc'
```

We often want to do this so that we can print a number:

```
my_dna = "ATGCGAGT"
```

```
dna_length = len(my_dna)
```

```
dna_length
```

```
8
```

```
# Different ways of printing
```

```
print("Length of the DNA sequence is " + str(dna_length))
```

```
Length of the DNA sequence is 8
```

or

```
print("Length of the DNA sequence is", dna_length)
```

```
Length of the DNA sequence is 8
```


📌 Converting between integers and strings : int()

Add a string to an integer: no, can't do that as they are different types

7 + "6"

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Add an integer to a string: no, can't do that as they are different types

"6" + 7

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: Can't convert 'int' object to str implicitly
```

int() will take a string and turn it into an integer

int("6") + 7

13

int("6.45") + 7

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ValueError: invalid literal for int() with base 10: '6.45'
```

int(float("6.45")) + 7

13

int(float("6.999999999999999")) + 7

13

int(float("6.999999999999999 9")) + 7

📌 Changing case

To turn a string into lower case, use a **method** called `lower()`

```
my_dna = "ATGC"
```

```
# Print my_dna in lower case
```

```
print(my_dna.lower())
```

```
atgc
```

```
"ABC4".lower()
```

```
'abc4'
```

What we need to know about lower()

- `lower()` is a **method**, not a function
- we write the name of the variable first, then a dot, then the method name
- `lower()` has no arguments
- `lower()` only works on strings, not numbers
- `lower()` doesn't change the variable, it returns a lower case version

Err.... what is the difference? [Link to discussion](#)

A function is a piece of code that is called by name. It can be passed data to operate (i.e. the parameters) and can optionally return data (the return value). All data passed to a function is explicitly passed.

A method is a piece of code that is called by a name that is associated with an object. In most respects it is identical to a function except for two key differences:

- 1. A method is implicitly passed the object on which it was called.*
- 2. A method is able to operate on data that is contained within the class (remember that an object is an instance of a class - the class is the definition, the object is an instance of that data).*

As we will see later, it will become clearer.... (promise/hope!)

Create the variable `my_dna`

`my_dna = "ATGC"`

Print the variable `my_dna`

`print("before: " + my_dna)`

before: ATGC

Apply the lower method and store the result

`lowercase_dna = my_dna.lower()`

`print(lowercase_dna)`

atgc

`print(my_dna.lower())`

atgc

`my_dna.lower()`

'atgc'

Print the variable `my_dna` **again: it's unchanged**

`print("after: " + my_dna)`

after: ATGC

🏠 Find and replace in strings

Use the `replace()` method:

```
# Make up a random protein sequence
```

```
protein = "vlspadktnv"
```

```
print(protein)
```

```
vlspadktnv
```

```
protein
```

```
'vlspadktnv'
```

```
# Replace valine (v) with tyrosine (y) from, to
```

```
print(protein.replace("v", "y"))
```

```
ylspadktny
```

```
# Replace more than one sequential character
```

```
print(protein.replace("vls", "ymt" ))
```

```
ymtpadktnv
```

```
# Replace more than one sequential character, no print command
```

```
protein.replace("vls", "ymt" )
```

```
'ymtpadktnv'
```

```
# Replace more than one sequential character, create new variable with the value
```

```
changed_protein = protein.replace("vls", "ymt" )
```

```
# The original variable is not affected
```

```
print(protein)
```

vlspadktnv
print(changed_protein)
ymtpadktnv

📌 Extracting part of a string

To get just part of a string, put square brackets `[ ]` after the variable name with the start and stop positions separated by a colon, e.g. `[2:23]`

Two very important things to know...

1. We start counting at **zero** rather than one
2. Positions are inclusive at the start and exclusive at the stop

```
# Make up a random protein sequence
```

```
# Get positions 3 to 6 of the "Python protein"
```

```
# This returns us amino acids 4, 5, and 6 of the "real protein"
```

```
vlspadktnv  
0123456789
```

```
protein = "vlspadktnv"
```

```
protein
```

```
'vlspadktnv'
```

```
protein[3:6]
```

```
'pad'
```

```
# To go to the end of the string, just leave out the stop position
```

```
protein[5:]
```

```
'dktnv'
```

This manipulation of variable values is called [slicing](#), and we will use it quite a lot.

📈 Counting substrings

The `count()` method will tell you how many times a short string occurs inside a longer one

```
# Make up a random protein sequence
```

```
protein = "vlspadktnv"
```

```
# Count specific amino acid residues in the protein
```

```
valine_count = protein.count('v')
```

```
lsp_count = protein.count('lsp')
```

```
tryptophan_count = protein.count('w')
```

```
# Now print the counts
```

```
print("valines: " + str(valine_count ))
```

```
valines: 2
```

```
print("lsp: " + str(lsp_count ))
```

```
lsp: 1
```

```
print("tryptophans: " + str(tryptophan_count ))
```

```
tryptophans: 0
```

📌 Finding positions of substrings

The `find()` method will tell you the (first) position of a short string inside a longer one.

The basic syntax is

```
find(str, beg=0, end=len(string))
```

where `beg` and `end` default to the first (begin) and last (end) positions of the string.

```
# Make up a random protein sequence
```

```
protein = "vlspadktnv"
```

```
protein.count('p')
```

```
1
```

```
print("The index of the first proline is " + str(protein.find('p' )))
```

```
The index of the first proline is 3
```

```
print(protein.find('kt' ))
```

```
6
```

```
vlspadktnv  
0123456789
```

```
# Look for something that we know isn't going to be found
```

```
print(protein.find('w' ))
```

```
-1
```

```
# Make up a random protein sequence
```

```
protein = "vlspadktnv"
```

```
# Make a variable to hold what we want to search for
```

```
residue = "s"
```

```
# Use the variable to search: this is the incorrect format, residue is a variable!!
```

```
residue.find(protein)
```

```
-1
```

```
# Use the variable to search: this is the correct format this time
```

```
protein.find(residue)
```

```
2
```

```
print("The amino acid", residue,  
      "...  "is at Python position", protein.find(residue),  
      "...  "in the short protein", protein)
```

```
The amino acid s is at Python position 2 in the short protein vlspadktnv
```

```
# Change the value of the variable "residue" to search for amino acid v
```

```
residue = "v"
```

```
protein.find(residue)
```

```
0
```

```
# We can use the same output line for the different variable values
```

```
print("The amino acid", residue, "is at Python position",  
      protein.find(residue), "in the short protein", protein)
```

```
The amino acid v is at Python position 0 in the short protein vlspadktnv
```

```
# Huh?! We can see 2 valines!
```

```
protein.count(residue)
```

```
2
```

```
# Remember, the basic syntax is
```

```
find(str, beg=0, end=len(string))
```

```
# Change the "search region" of the string: there was more than one "find"
```

```
protein.find(residue,0,)
```

```
0
```


protein.find(residue,1,)

9

We'll deal with "multiple finds" later on when we do regular expressions...

📌 Summary of things!

Types of things: data types

- strings are always surrounded by quotes
- numbers are always written without quotes

Other tools

- `+` can add numbers together (also `/` will divide, `*` will multiply and `-` will subtract)
- `[1:2]` can get parts of a string
- `\n` means start a new line, `\t` means a tab

Parts of code

- function
- method
- arguments
- statement
- quotes
- comments
- variables

Functions

- `print()` for showing text on the screen
- `len()` for finding the length of a string
- `str()` for converting a number to a string
- `int()` for converting a string to a number

String methods

- `upper()` for changing to upper case
- `replace()` for changing parts of a string
- `count()` for counting substrings
- `find()` for locating substrings

Methods, functions ???

A common question is "how do they differ, or is it just semantics"?

Many answers out there, some rather more complicated than necessary...!

Good clear answer:

<https://stackoverflow.com/questions/3786881/what-is-a-method-in-python>

With some modifications: In Python, a method is essentially a function that is available for a given object because of the object's type.

For example, if we create a list object (we'll be doing this very soon!):

```
my_list = [1, 2, 3]
```

the `append()` method can be applied to `my_list` because it is a Python list.

```
my_list.append(4)
```

```
my_list
```

```
[1, 2, 3, 4]
```

All lists have an `append()` method simply because they are lists: "because they do".

As another example, if we create:

```
my_string = 'some lowercase text'
```

the `upper()` method can be applied to `my_string` simply because it's a Python string.

```
my_string.upper()
```

```
'SOME LOWERCASE TEXT'
```

Lists don't have an `upper()` method
Strings don't have an `append()` method

Why? Because they just don't!

Methods only exist for a particular object if they have been explicitly defined for that type of object, and Python's developers have (so far) decided that those particular methods are needed for those particular objects.

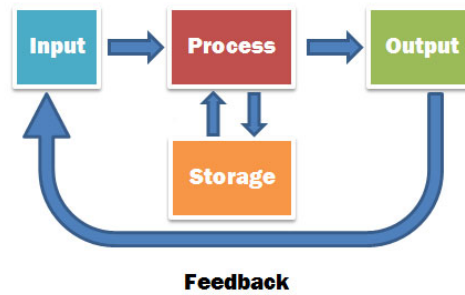
To call a method, the format is

`object_name.method_name()`

and any arguments to the method are listed within the parentheses.

The method implicitly acts on the object being named, and thus some methods don't have any stated arguments since the object itself is the only necessary argument.

For example `my_string.upper()` doesn't have any listed arguments because the only required argument is the object itself, `my_string`.



[Log in](#)

- set your alias to p3 (only if you think it would be useful...!)
- break problems down into small chunks if possible and test them
- try to solve amongst yourselves before looking at solutions
- don't worry if you don't finish
- Google is your friend
- remember that Python starts counting at 0...!

Let's try and be tidy...

Make a directory

```
mkdir ~/Exercises/Lecture11
```

```
cd ~/Exercises/Lecture11
```

Starting python with our alias

```
p3
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

exit() or Ctrl-D to exit Python

help

Type `help()` for interactive help, or `help(object)` for help about `object`.
>>>

help()

Welcome to Python 3.12's help utility! If this is your first time using Python, you should definitely check out the tutorial at <https://docs.python.org/3.12/tutorial/>.

Enter the name of any module, keyword, or topic to get help on writing Python programs and using Python modules. To get a list of available modules, keywords, symbols, or topics, enter "modules", "keywords", "symbols", or "topics".

Each module also comes with a one-line summary of what it does; to list the modules whose name or summary contain a given string such as "spam", enter "modules spam".

To quit this help utility and return to the interpreter, enter "q" or "quit".

help>

q to exit the help

1. 📌 Calculating A+T content

Here's a short DNA sequence:

ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTCAT

Write a program using the

- [vi \(vi cheatsheet\)](#)
- [nano \(nano cheatsheet\)](#)

or... plain text editor, saving the file in the directory you created that will print out to the screen the A+T content of this DNA sequence.

Don't forget that you can have two or more MobaXterm/terminal windows open at the same time....

Your program will have to:

1. count the number of A nucleotides
2. count the number of T nucleotides
3. add them up and divide by the length of the sequence

Hint: remember that you can use normal mathematical symbols like add (+), subtract (-), multiply (*), divide (/) and parentheses to carry out calculations on numbers in Python.

2. 📌 **Complementing DNA**

Here's a short DNA sequence:

ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTCAT

Write a program that will print the complement of this sequence. (Not the reverse complement! If you are unsure of DNA base-pairing, please ask!)

3. 📌 **Restriction fragment lengths**

Here's a short DNA sequence:

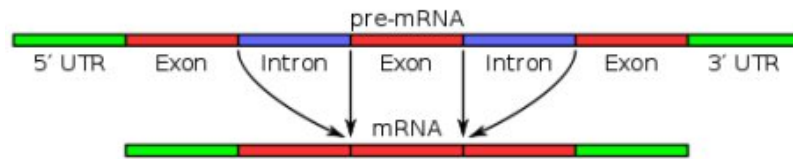
ACTGATCGATTACGTATAGTAGAATTCTATCATACATATATATCGATGCGTTCAT

The sequence contains a recognition site for the *EcoRI* restriction enzyme, which cuts at the motif **G*AATTC** (the position of the cut is indicated by an asterisk). Write a program which will calculate the size of the two fragments that will be produced when the DNA sequence is digested with *EcoRI*.

Your program will have to:

1. find the position of the motif in the sequence
2. use that to figure out the position of the cut
3. calculate the length of the second fragment based on the cut position

4. 🏠 Splicing out introns



Here's a short section (123 bases) of genomic DNA:

```
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCT
ACTCGATCGATCGATCGATCGATCGATCGATCGATCGATCATGCT
ATCATCGATCGATATCGATGCATCGACTACTAT
```

It comprises two exons and an intron. The first exon runs from the start of the sequence to the sixty-third real character, and the second exon runs from the ninety-first real character to the end of the sequence. Write a program that will print just the coding regions of the DNA sequence.

Your program will have to:

1. extract the first and second exon based on the coordinates and store these as variables
2. join together the two exons to make the coding sequence

Write a program that will calculate what percentage of the DNA sequence is coding.

Your program will have to:

1. figure out the length of the coding sequence
2. divide by the length of the original sequence and multiply by 100

Write a program that will print out the original genomic DNA sequence with coding bases in uppercase and non-coding bases in lowercase.

Your program will have to:

1. store the exons and intron separately as variables
2. convert the intron to lower case
3. concatenate the exons and intron to make one long sequence

Possible answers

1. Calculating A+T content

```
# Have a look at the sequence, do an estimate of the A+T content visually:  
maybe somewhere around 50% (0.5) or so?
```

```
# We need to store the information we need - the A and T count, and the  
length of the DNA sequence
```

```
my_dna =  
"ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTTCAT"  
length = len(my_dna)  
a_count = my_dna.count('A')  
t_count = my_dna.count('T')
```

```
# We can print these out to check that they look OK
```

```
print("length: " + str(length))  
length: 54  
print("A count: " + str(a_count))  
A count: 16  
print("T count: " + str(t_count))  
T count: 21
```

```
# Now let's try the calculation
```

```
at_content = a_count + t_count / length  
print("A+T content is " + str(at_content))  
A+T content is 16
```

This is definitely not right, it should be between zero and one!

Forgot the brackets, so the order of doing things was "wrong"

```
at_content = (a_count + t_count) / length
```

```
print("A+T content is " + str(at_content))
```

A+T content is 0.685185185185

"Final" version 1 including the "shebang line" at the top

```
#!/usr/bin/python3
```

Program to take a DNA sequence and calculate A+T content

Written by s123456 on 21 Oct 2025

```
#-----#
```

```
my_dna =
```

```
"ACTGATCGATTACGTATAGTATTTGCTATCATACATATATCGATGCGTTTCAT"
```

```
length = len(my_dna)
```

```
a_count = my_dna.count('A')
```

```
t_count = my_dna.count('T')
```

```
at_content = (a_count + t_count) / length
```

```
print("A+T content is " + str(at_content))
```

A+T content is 0.685185185185

"Final" version 2, shorter, and converted to percentage, as requested

```
#!/usr/bin/python3
```

Program to take a DNA sequence and calculate A+T content

Written by s123456 on 21 Oct 2025

```
# ----- #
```

Input

```
my_dna =
```

```
"ACTGATCGATTACGTATAGTATTTGCTATCATACATATATCGATGCGTTTCAT"
```

Process

```
at_content = (my_dna.count('A') + my_dna.count('T')) / len(my_dna)
```

Output

```
print("A+T content is",str(int(100*at_content)),"percent")
```

A+T content is 68 percent

Cool that works, so if we had saved our script as `Lect_11.script1.py` we could do...

```
# Back in the ordinary terminal (no longer in python3 session!)
```

```
python3 Lect_11.script1.py
```

```
A+T content is 68 percent
```

OR

```
chmod 755 Lect_11.script1.py
```

```
./Lect_11.script1.py
```

```
A+T content is 68 percent
```

2. Complementing DNA



The "obvious" thing doesn't work, as when you change A to T, then change T to A

```
# Need a sequential replacement approach as we haven't done loops or  
regular expressions yet!
```

```
# Could use the complement lower case, or numbers, or anything unique
```

```
my_dna =
```

```
"ACTGATCGATTACGTATAGTATTGCTATCATACATATATCGATGCGTTTCAT"
```

```
replacement1 = my_dna.replace('A', 't')
```

```
print(replacement1)
```

```
tCTGtTCGtTTtCGTtTtGTtTTTGCTtTCtTtCtTtTtTCGtTGCGTTcTt
```

```
replacement2 = replacement1.replace('T', 'a')
```

```
print(replacement2)
```

```
tCaGtaCGtaatCGatatGataaaGCataCtatCtatatataCGtaGCGaaCta
```

```
replacement3 = replacement2.replace('C', 'g')
```

```
print(replacement3)
tCaGtaCGtaatCGatatGataaaGCataCtatCtatatataCGtaGCGaaCta
replacement4 = replacement3.replace('G', 'c')
print(replacement4)
tgactagctaatgcatatcataaacgatagtagtatgtatatatagctacgcaagta
print(replacement4.upper())
TGACTAGCTAATGCATATCATAAACGATAGTATGTATATATAGCTACGCAAGTA
```

"Final" version 1

```
#!/usr/bin/python3
# Program to take a DNA sequence and complement it
# Written by s123456 on 21 Oct 2025
#-----#

my_dna =
"ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTCAT"
replacement1 = my_dna.replace('A', 't')
replacement2 = replacement1.replace('T', 'a')
replacement3 = replacement2.replace('C', 'g')
replacement4 = replacement3.replace('G', 'c')
print("The complement sequence is " + str(replacement4.upper()))
The complement sequence is
TGACTAGCTAATGCATATCATAAACGATAGTATGTATATATAGCTACGCAAGTA
```

"Final" version 2: single line!

```
#!/usr/bin/python3
# Program to take a DNA sequence and complement it
# Written by s123456 on 21 Oct 2025
#-----#
# Input
my_dna =
"ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTCAT"
# Process AND output
print("The complement sequence is", my_dna.replace('A',
't').replace('T', 'a').replace('G', 'c').replace('C', 'g').upper())
The complement sequence is
TGACTAGCTAATGCATATCATAAACGATAGTATGTATATATAGCTACGCAAGTA
```

0	1	2	3	4	5
0123456789012345678901234567890123456789012345678901234					
ACTGATCGATTACGTATAGTAGAATTC	TATC	ATAC	ATATATATCGATGCGTTCA		

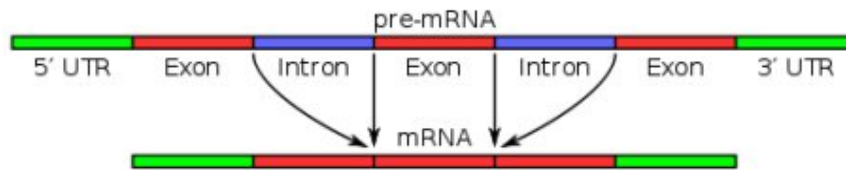
```
site = "GAATTC"
```



```
print("Lengths of",site,"cleaved fragments are",my_dna.find(site) +  
1,"and",len(my_dna) - (my_dna.find(site) + 1),"bases")
```

Lengths of GAATTC cleaved fragments are 22 and 33 bases

4. Splicing out introns



Here's a short section (123 bases) of genomic DNA:

```
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCT  
ACTCGATCGATCGATCGATCGATCGATCGATCGATCGATCATGCT  
ATCATCGATCGATATCGATGCATCGACTACTAT
```

The sequence comprises two exons and an intron. The first exon runs from the start of the sequence to the sixty-third character, and the second exon runs from the ninety-first character to the end of the sequence.

The sequence was:

```
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCT  
ACTCGATCGATCGATCGATCGATCGATCGATCGATCGATCATGCT  
ATCATCGATCGATATCGATGCATCGACTACTAT
```

First, store the DNA and extract the exons as separate variables:

```
my_dna  
="ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGATCGATCGATCG
```

Remember that we start counting from zero, and that positions are inclusive at the start and exclusive at the end

```
exon1 = my_dna[0:63]
```

```
exon2 = my_dna[90:]
```

```
### Exons joined ###
```

```
coding_length = len(exon1 + exon2)
```

```
total_length = len(my_dna)
```

```
print("### Coding percentage (rounded) ###\n" + str(int((coding_length /
total_length) * 100)))
```

```
# We already specified the exons
```

```
exon1 = my_dna[0:63]
```

```
exon2 = my_dna[90:]
```

```
intron = my_dna[63:90]
```

```
print("### Exon intron exon ###\n" + exon1 + intron.lower() + exon2)
```

CTRL-D to exit

cat Lect_11.script exons.py

```
#!/usr/bin/python3
```

```
my dna ="ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGATCGATCGATCGATCGATC
```

```
# Remember that we start counting from zero, and that positions are inclusive at the start
exon1 = my_dna[0:63]
exon2 = my_dna[90:]
```

```
# Now we can just join the two exons and print them out
print("### Exons joined ###\n" + exon1 + exon2)
```

```
# To calculate the coding percentage, we just
# have to take the length of the exons,
# divide by the length of the sequence, and multiply by 100
coding_length = len(exon1 + exon2)
total_length = len(my_dna)
print("### Coding percentage (rounded) ###\n" + str(int((coding_length / total_length) * 100)))
```

```
# To print out the upper/lower case version,
# we need to take the middle bit
# i.e. the intron and convert it to lower case,
# then join the bits back together
# We already specified the exons
exon1 = my_dna[0:63]
exon2 = my_dna[90:]
intron = my_dna[63:90]
print("### Exon intron exon ###\n" + exon1 + intron.lower() + exon2)
```

Let's try it out!

```
chmod 755 ./Lect_11.script_exons.py
```

```
./Lect_11.script_exons.py
```

```
### Exons joined ###  
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGAATCATCGATCGATATCGATGCATCGAC  
### Coding percentage (rounded) ###  
78  
### Exon intron exon ###  
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGAtcgcgtcgatcgatcgatcgatcgatcgatcgct
```

Let's try it out from within Python3 (we will cover this in later lectures)

p3

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
exec(open('Lect_11.script_exons.py').read())
```

```
### Exons joined ###
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGAATCATCGATCGATATCGATGCATCGAC
### Coding percentage (rounded) ###
78
### Exon intron exon ###
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGAatcgatcgatcgatcgatcgatcgatcgat
```

CTRL-D to exit

📦 git and GitHub time!

... remember them ...?!

If you are having issues with using/manipulating git branches, this is a particularly good website:

<https://www.nobledesktop.com/learn/git/git-branches>

Depending on whether you have added your script files from Lectures 08 (eddie) and 09 (MySQL), this might/will be different...

```
git st
```

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
deleted:    ../Lecture04/CodeFiles.tar.gz
modified:   ../Lecture06/run_this_script_v1
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
../BN.out
../BN.sh
../Lecture04/Als_ICA/
../Lecture04/all_the_things_I_did
../Lecture04/outs/
../Lecture05/Country.Mozambique.details
../Lecture07/Cosmoscarta.nuc.fa
../Lecture07/Cosmoscarta.nuc.gis
../Lecture07/myfileofaccessions.txt
./
```

no changes added to commit (use "git add" and/or "git commit -a")

git add *.py

git commit -m "My python scripts from the first BPSM python lecture"

```
[master 6d6f5e7] My python scripts from the first BPSM python lecture
1 file changed, 24 insertions(+)
create mode 100755 Lecture11/Lect_11.script_exons.py
```

git log

```
6d6f5e7 (HEAD -> master) My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lecture06/Lec_6_branch, Lec_6_branch) Kept bash script output files from Lecture
f8e5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Let's make yet another NEW repository on GitHub for all our lecture exercises

We will need to already have our details as variables...

```
curl -u ${mypersonalGitHub_username}:${mypersonalgittoken} \
https://api.github.com/user/repos -d '{"name":"BPSM_LectureStuff"}'
```

```
{
  "id": 554463300,
  "node_id": "R_kgDOIQxwRA",
  "name": "BPSM_LectureStuff",
  "full_name": "B123456-2121/BPSM_LectureStuff",
  "private": false,
  "owner": {
    ...
```

**# Tell git about our repository on GitHub, and that
we are calling the link "AllLectures" (or something!)**

git remote add AllLectures

[https://github.com/\\${mypersonalGitHub_username}/BPSM_LectureStuff.git](https://github.com/${mypersonalGitHub_username}/BPSM_LectureStuff.git)

Push our local repository to the new repo we have just created at GitHub

git push AllLectures master

```
Username for 'https://github.com': B123456-2121
Password for 'https://B123456-2121@github.com': xxxxxx
Enumerating objects: 120, done.
Counting objects: 100% (120/120), done.
Delta compression using up to 256 threads
Compressing objects: 100% (113/113), done.
Writing objects: 100% (120/120), 26.60 MiB | 8.86 MiB/s, done.
Total 120 (delta 35), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (35/35), done.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
 * [new branch]      master -> master
```

git log

```
6d6f5e7 (HEAD -> master, AllLectures/master) My python scripts from the first BPSM python
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lecture06/Lec_6_branch, Lec_6_branch) Kept bash script output files from Lecture
f8e5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

If you want to delete a repository on GitHub, just make sure that you have the correct scopes (permissions) set up for your token.

How to do this is shown on [our GitHub account instructions page](#).

```
# Let's remove the BPSM_Lecture06 repository from GitHub
```

```
curl -u ${mypersonalGitHub_username}:${mypersonalgittoken} -X  
DELETE
```

```
https://api.github.com/repos/${mypersonalGitHub_username}/BPSM_Lectu
```

```
# What git remote links do we have?
```

```
git remote
```

```
AllLectures  
Lecture06  
Lecture07  
origin
```

```
# We don't need Lecture06 any more!
```

```
git remote rm Lecture06
```

```
# What git remote links do we have NOW!?
```

```
git remote
```

```
AllLectures  
Lecture07  
origin
```



Welcome to Python!

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!



Sources used